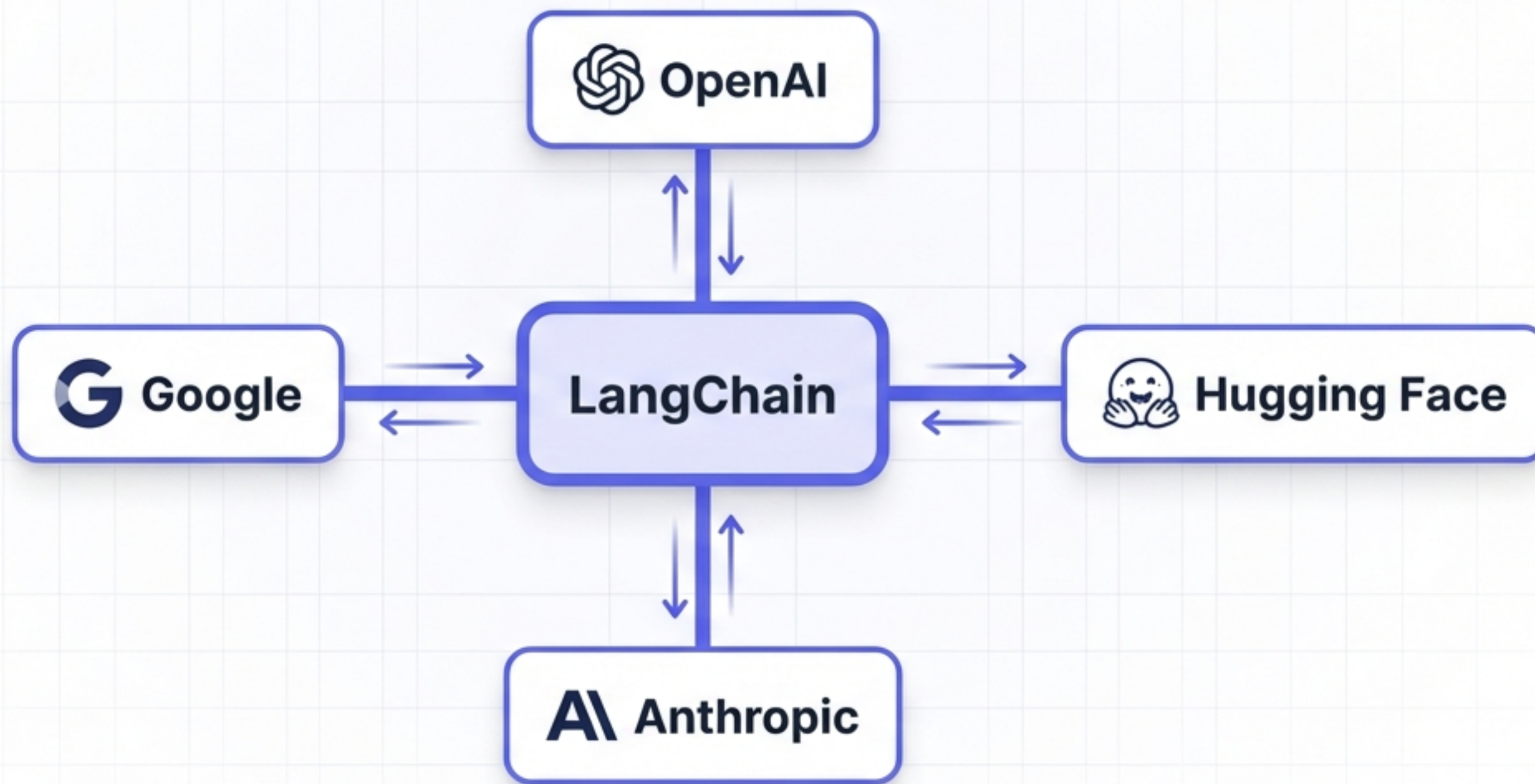


Mastering LangChain Models

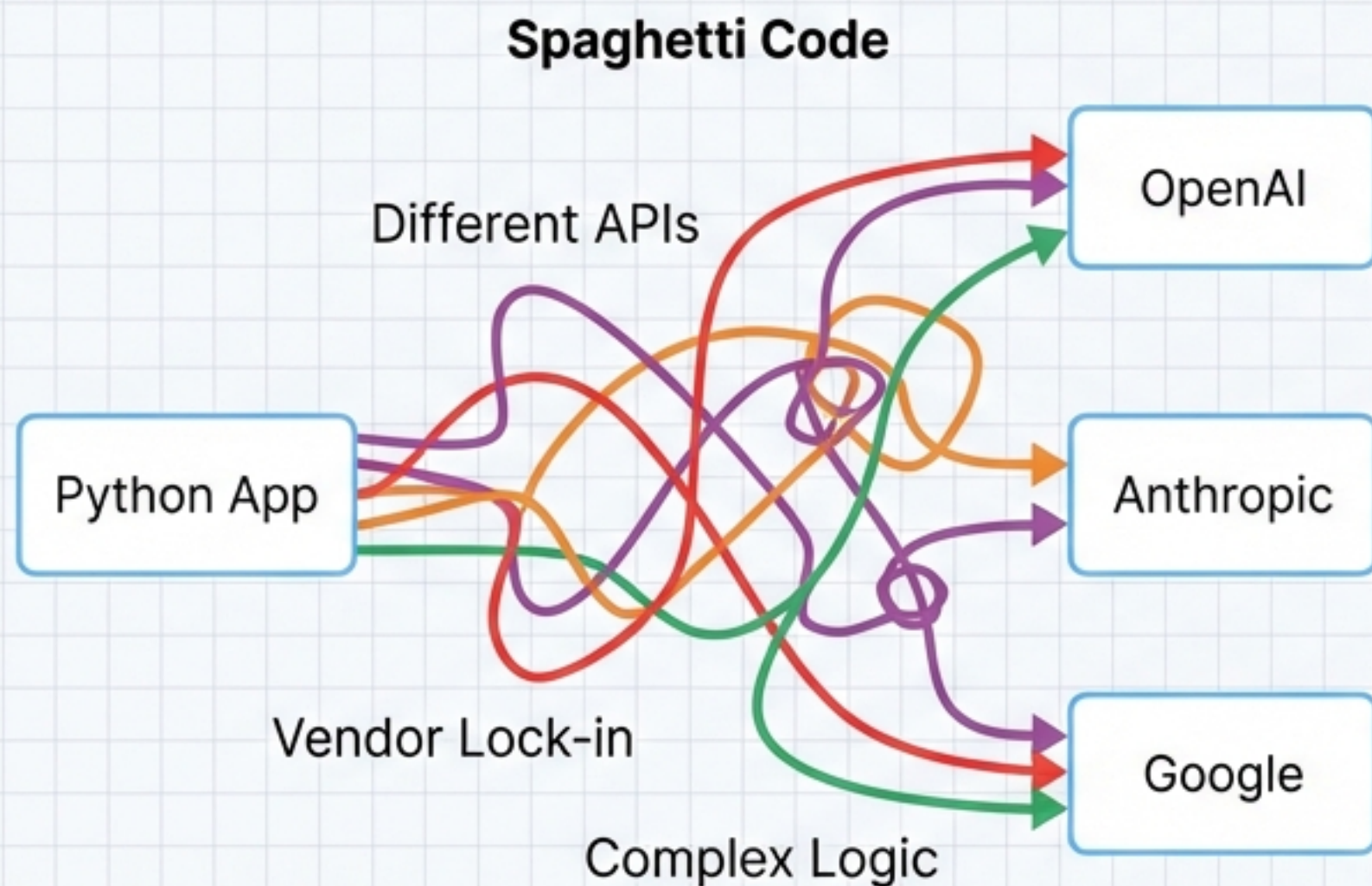
A Deep Dive into Language Models, Embeddings, and Open Source Implementations



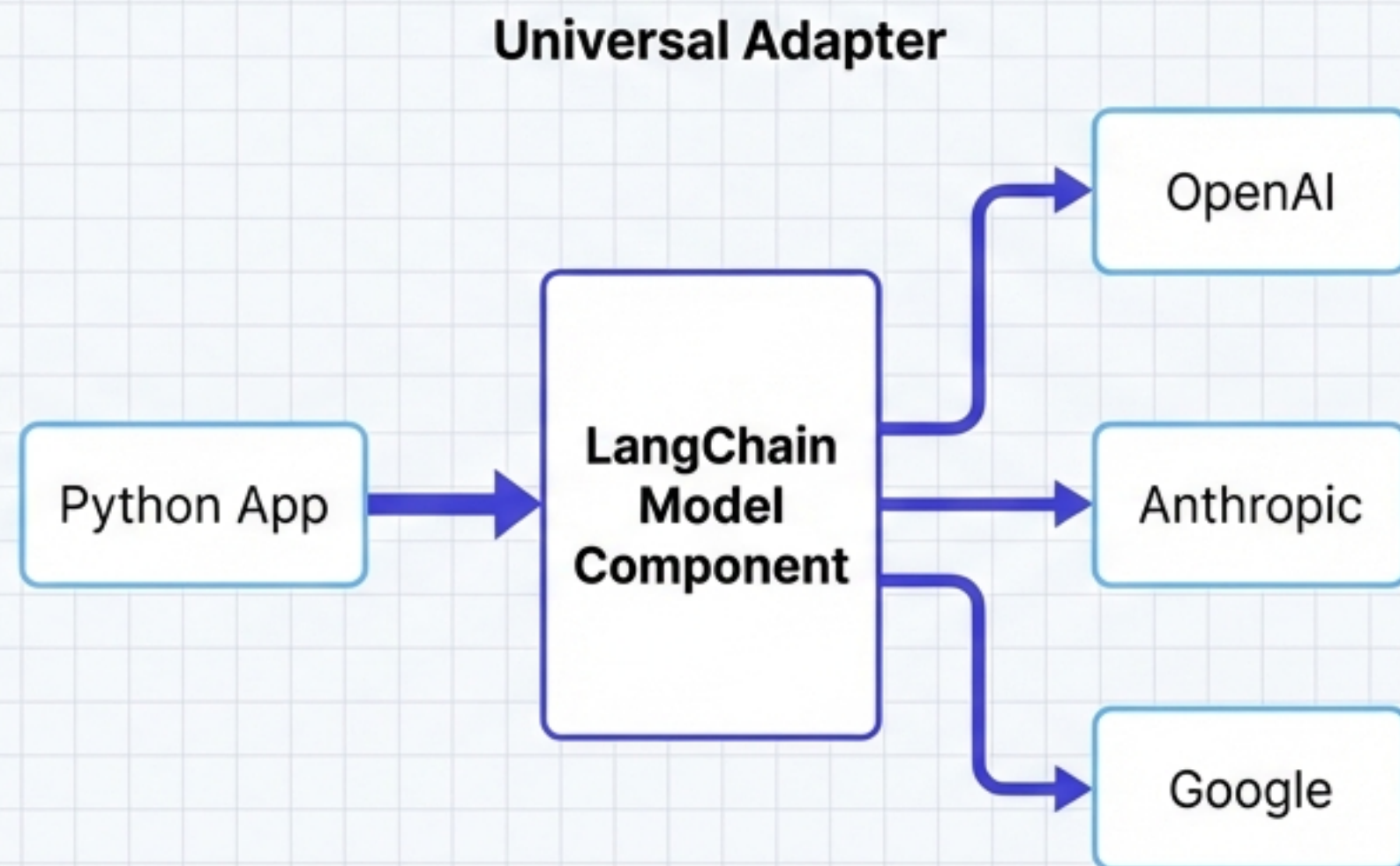
Based on the LangChain Models tutorial by CampusX

The Universal Interface

The Problem: Fragmentation

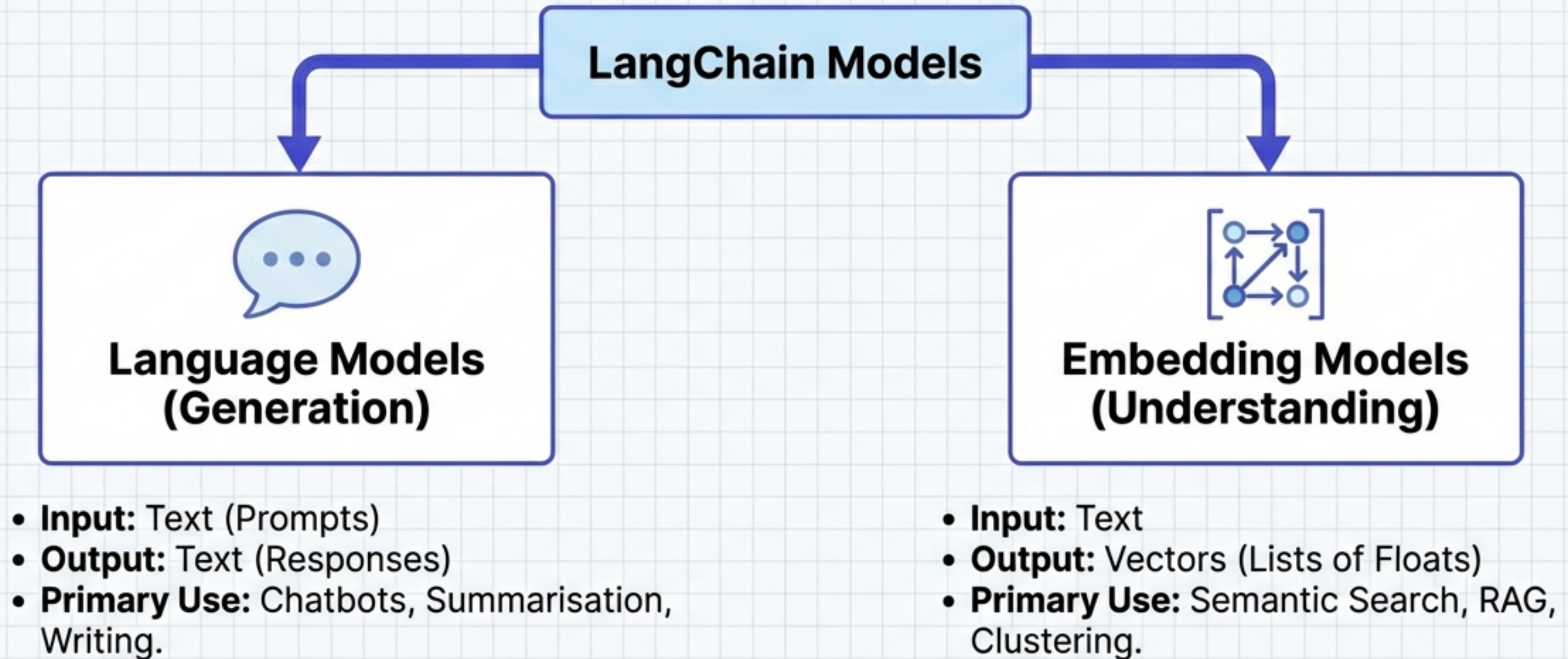


The Solution: LangChain



Standardized Interface: Switch models with minimal code changes.

The Model Taxonomy

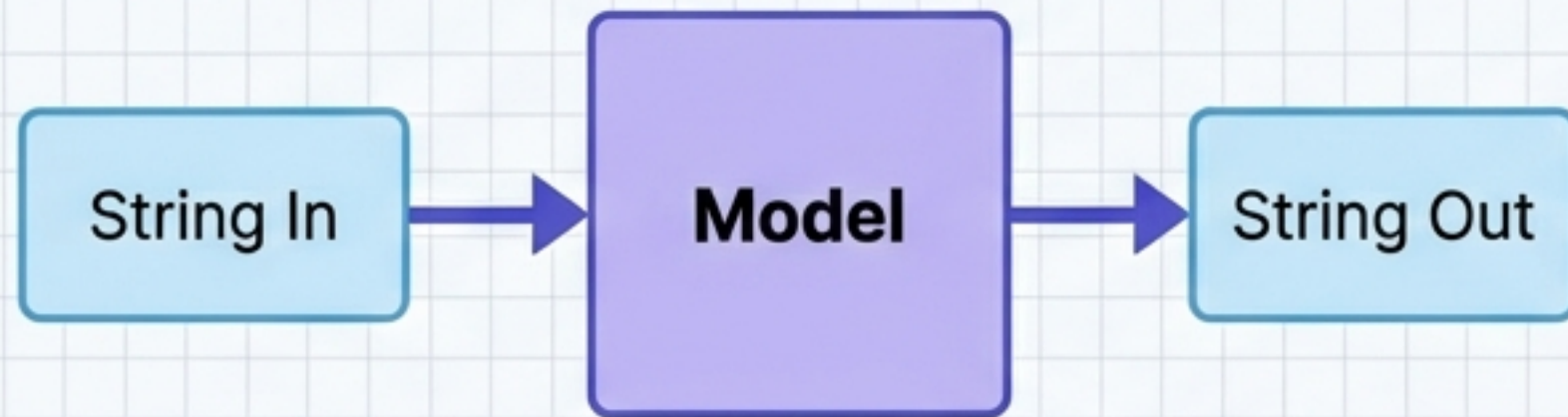


Insight: Language models generate content. Embedding models calculate mathematical relationships.

Evolution: LLMs vs. Chat Models

LLMs (Legacy)

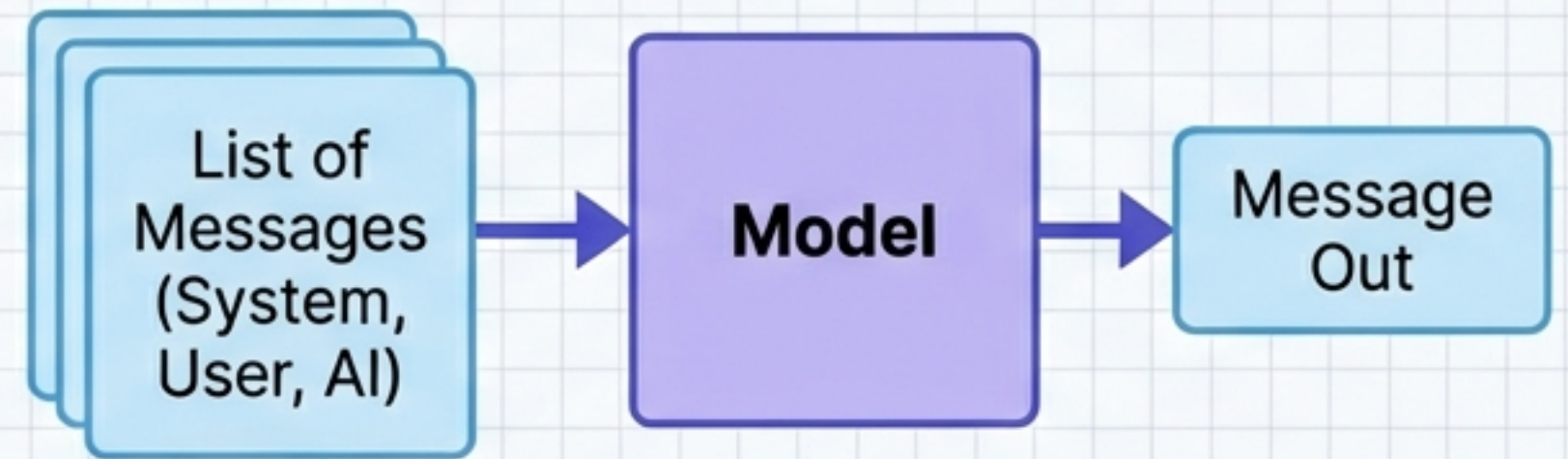
The Raw Text Predictor



- **Stateless:** No memory of past turns.
- **Class:** BaseLLM
- **Status:** Deprecated for new projects.

Chat Models (The Standard)

The Conversational Engine



- **Structured:** Handles Roles (System/User).
- **Class:** BaseChatModel
- **Status:** Recommended standard.

Note: LangChain is moving away from raw LLMs towards Chat Models for all GenAI applications.

The Unified Syntax: Closed Source Providers

```
from langchain_openai import ChatOpenAI  
model = ChatOpenAI(model='gpt-4')
```

```
from langchain_anthropic import ChatAnthropic  
model = ChatAnthropic(model='claude-3-opus')
```

```
from langchain_google_genai import ChatGoogleGenerativeAI  
model = ChatGoogleGenerativeAI(model='gemini-1.5-pro')
```

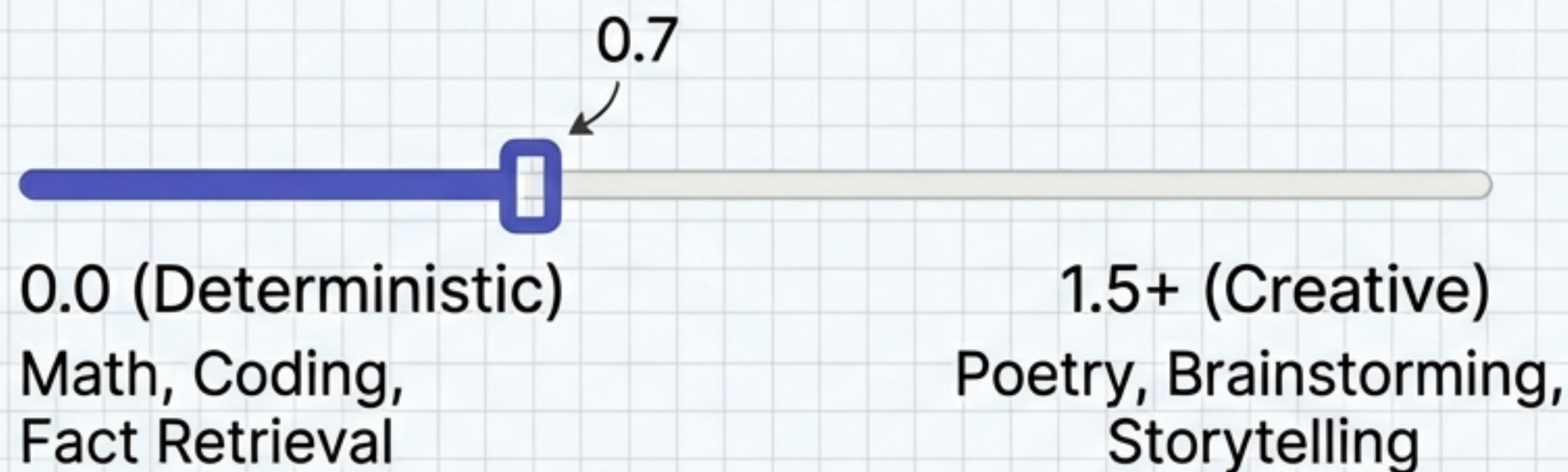
Requires API keys
in .env file.

The Universal Method:

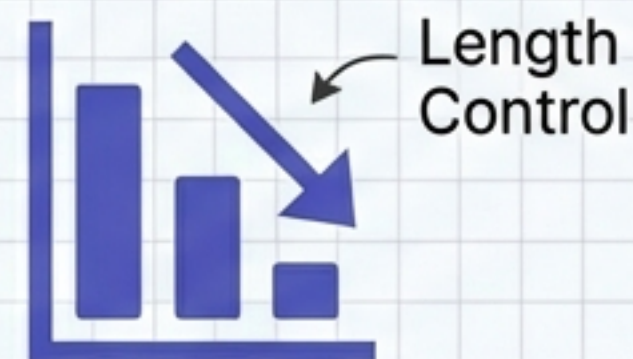
```
response = model.invoke('What is the capital of India?')
```

Controlling the Output

Temperature (Creativity Slider)



Max Tokens



Limits response verbosity
and controls API costs.

```
model = ChatOpenAI(  
    model='gpt-4',  
    temperature=0.5, # Balanced creativity  
    max_tokens=100  # Limit output length  
)
```

Closed Source vs. Open Source

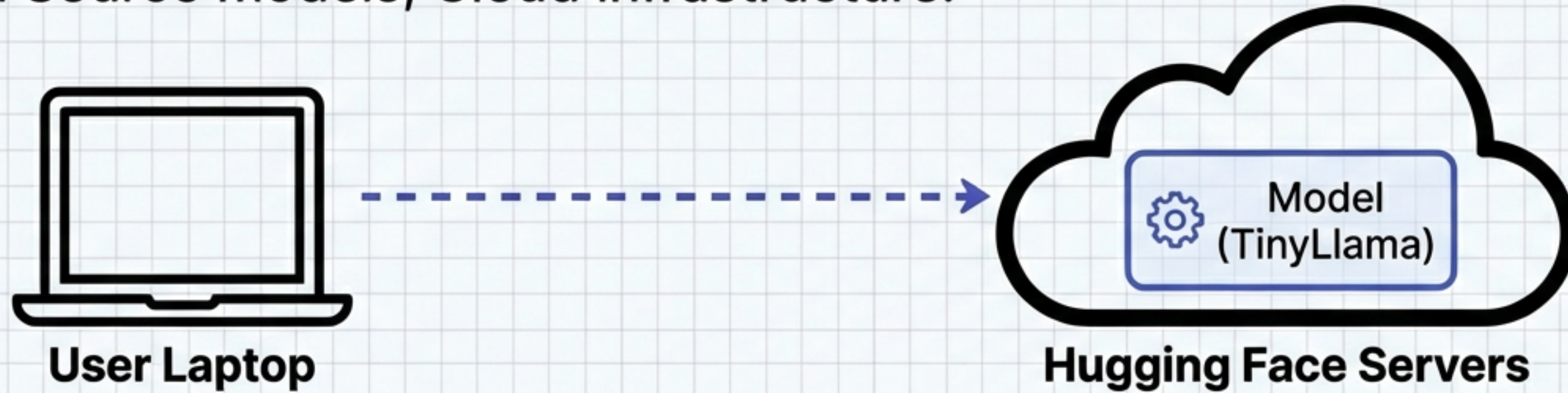
Feature	Closed Source (e.g., OpenAI)	Open Source (e.g., Llama)
Cost	Paid (Per Token)	Free (Mostly)
Privacy	Data sent to external servers	Data stays local (Full Privacy)
Control	Zero control over weights	Full control (Fine-tuning)
Hardware	No requirement (Cloud)	Requires High RAM / GPU



The Hub: Hugging Face hosts thousands of open models like Llama-3, Mistral, and TinyLlama.

Method 1: Hugging Face Inference API

Open Source models, Cloud infrastructure.



Pros

- Instant setup, No GPU needed.

Cons

- Rate limits, Data leaves local machine.

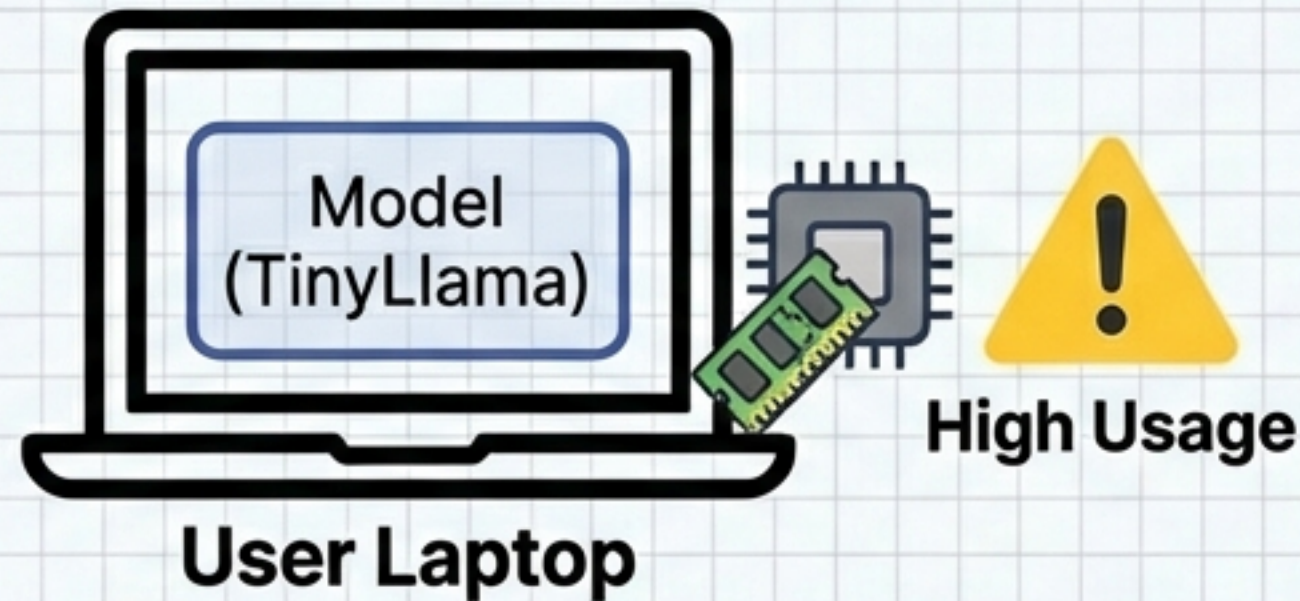
```
from langchain_huggingface import HuggingFaceEndpoint

llm = HuggingFaceEndpoint(
    repo_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    task='text-generation'
)

llm.invoke('Who is Virat Kohli?')
```

Method 2: Running Models Locally (Pipelines)

Total privacy and control.



Pros

- Offline, 100% Private, Fine-tunable.

Cons

- Heavy hardware usage (RAM/GPU intensive).

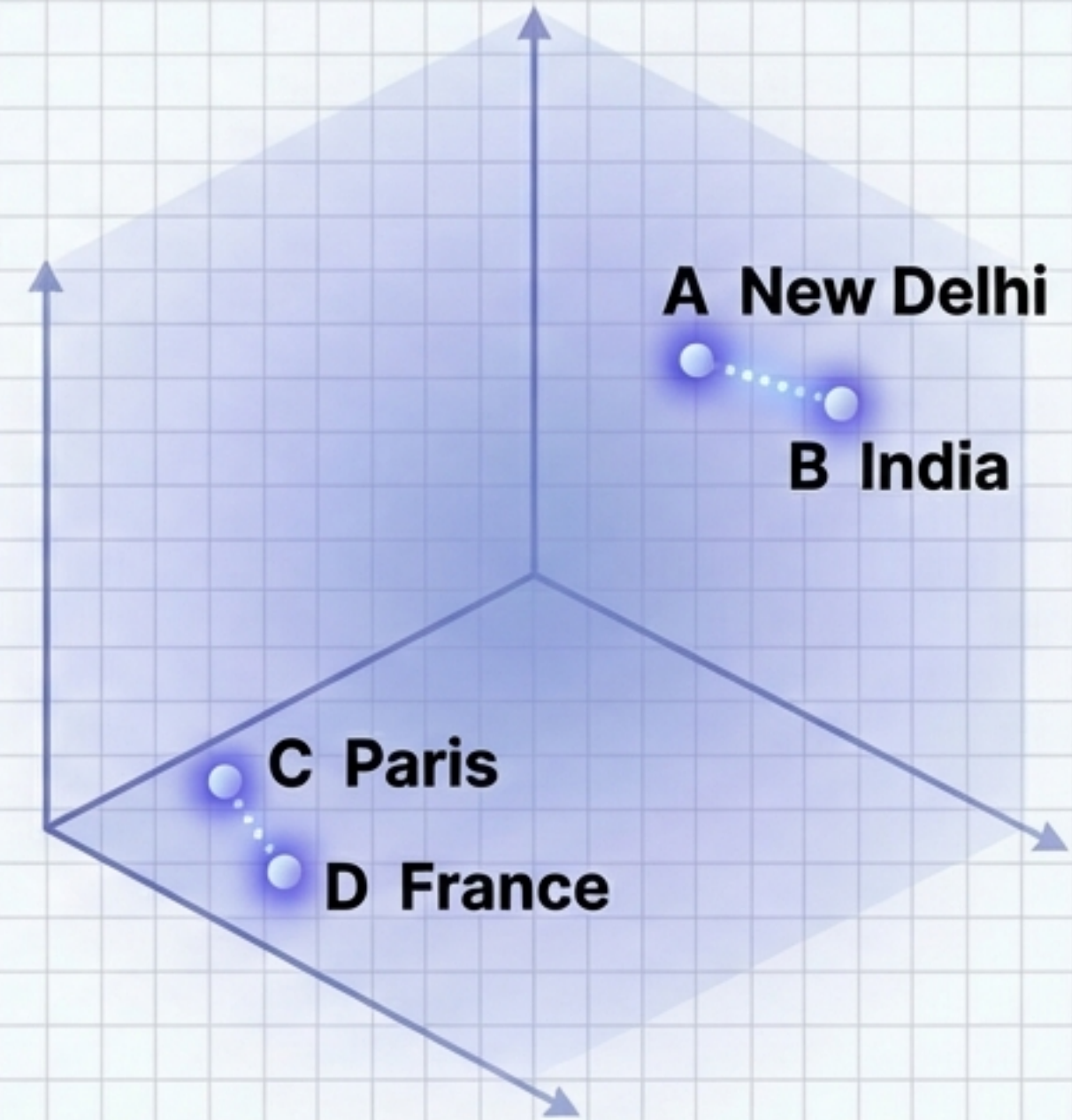
```
from langchain_huggingface import HuggingFacePipeline

# Downloads model weights to local cache
llm = HuggingFacePipeline.from_model_id(
    model_id='TinyLlama/TinyLlama-1.1B-Chat-v1.0',
    task='text-generation',
    pipeline_kwargs={'max_new_tokens': 100}
)
```

Understanding Embeddings

Concept: Converting text into Vectors (Lists of Numbers).

Why? Computers can't read text, but they can calculate the mathematical distance between numbers.



Delhi is the capital of India → $[-0.012, 0.453, -0.231, \dots]$

Generating Embeddings: The Code

OpenAI (Paid/High Quality)

Model: text-embedding-3-large

```
embeddings = OpenAIEmbeddings()
```

Hugging Face (Free/Local)

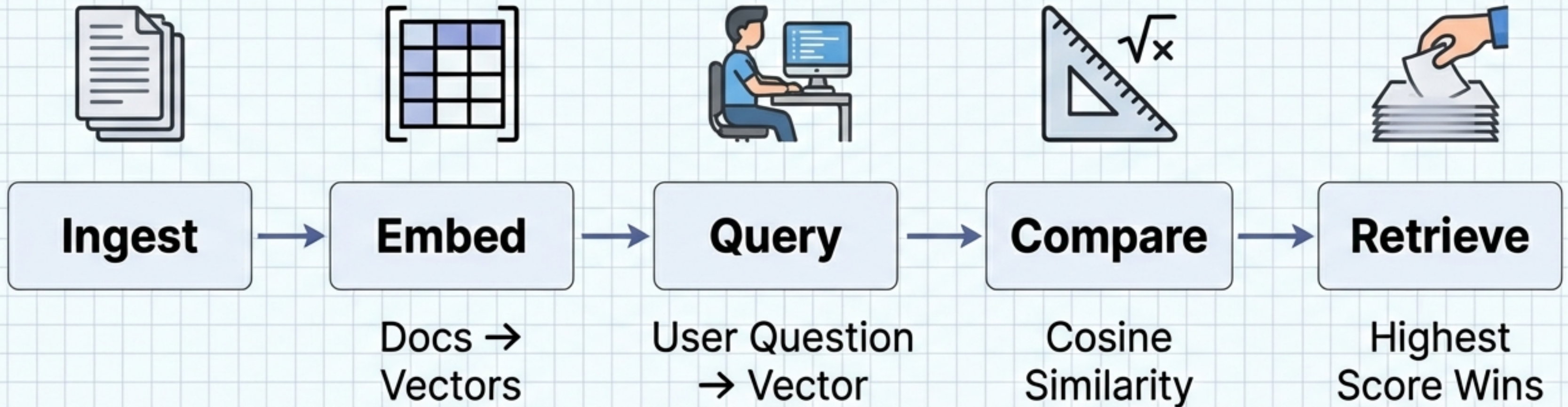
Model: sentence-transformers/all-MiniLM-L6-v2

```
embeddings = HuggingFaceEmbeddings(  
    model_name='all-MiniLM-L6-v2'  
)
```

Key Methods

- **embed_query(text)**: For single user questions.
- **embed_documents([list])**: For batch processing document sets.

Application: Document Similarity Search



Goal: Build a system where a user asks '**Tell me about Virat Kohli**' and the AI retrieves the specific document about him from a dataset.

The Code: Building the Engine

```
from sklearn.metrics.pairwise import cosine_similarity
```

Requires
scikit-learn

```
# 1. Embed the Data
```

```
doc_vectors = embeddings.embed_documents(documents)
```

```
query_vector = embeddings.embed_query('Tell me about Virat Kohli')
```

Must use
same model
for both

```
# 2. Calculate Similarity
```

```
# Note: Cosine similarity measures the angle between vectors
```

```
scores = cosine_similarity([query_vector], doc_vectors)
```

```
# 3. Retrieve
```

```
# Logic finds the index of the highest score
```

The Result: Semantic Matching

Output

> User Query: 'Tell me about Virat Kohli'

[MATCH FOUND] Score: 0.82

Document: 'Virat Kohli is an Indian cricketer known for his aggressive approach...'

[NO MATCH] Score: 0.12

Document: 'Paris is the capital of France.'

Takeaway: The model successfully matched the query to the cricketer document based on *meaning*, **despite no exact keyword overlap, while correctly ignoring the irrelevant document about Paris.**

Summary & Next Steps

Chat Models

Use for generation.
Prefer ChatOpenAI or ChatAnthropic.

Embeddings

Use for
search/understanding.
all-MiniLM is a great
free starter.

Open Source

Offers control/privacy
but demands hardware
resources.

The Road Ahead

→ Next: We manually built a retrieval system. The next step is scaling this with Vector Databases to build full RAG Applications.

Prerequisites: `pip install langchain-openai langchain-huggingface scikit-learn`